

```
main = do
  putStrLn "Enter your name:"
  name <- getLine
  putStrLn ( "Hello " ++ name)
```

Executing the code in GHCi gives the following results:

```
ghci> main
Enter your name :
Shakthi
Hello Shakthi
```

The *putStr* function is similar to the *putStrLn* function, except that it doesn't emit the new line after printing the output string. Its type signature and an example are shown below:

```
ghci> :t putStr
putStr :: String -> IO ()

ghci> putStr "Alpha "
Alpha ghci>
```

The *putChar* function takes a single character as input, and prints the same to the output. For example:

```
ghc> :t putChar
putChar :: Char -> IO ()

ghci> putChar 's'
s
```

The *getChar* function is similar to the *getLine* function, except that it takes a *Char* as input. Its type signature and usage are illustrated below:

```
ghci> :t getChar
getChar :: IO Char

ghci> a <- getChar
d

ghci> a
'd'

ghci> :t a
a :: Char
```

The *print* function type signature is as follows:

```
ghci> :t print
print :: Show a => a -> IO ()
```

It is a parameterised function, which can take an input of any type that is an instance of the *Show* type class and prints

that to the output. Some examples are given below:

```
ghci> print 1
1

ghci> print 'c'
'c'

ghci> print "Hello"
"Hello"

ghci> print True
True
```

The *getContents* function reads the input until the end-of-file (EOF) and returns a string. Its type signature is shown below:

```
ghci> :t getContents
getContents :: IO String
```

An example of code is demonstrated below. It only outputs lines whose length is less than five characters:

```
main = do
  putStrLn "Enter text:"
  text <- getContents
  putStr . unlines . filter (\line -> length line < 5) $
  lines text
```

Testing the above example gives the following output:

```
ghci> main
Enter text:
a
a

it
it

the
the

four
four

empty

twelve

haskell
```

```
o
o
```